Assignment 2

10 pts



CSE Department
Computer Vision CMAI 480

Dr. Khalil Yousef

Lab Instructions:

- The assignment must be turned electronically using Blackboard.
- This is an individual assignment -- each student should submit their own solution.
- In every uploaded file, include your name, date, and the assignment name.
- Untested code is buggy. Your code should run when I test it (or you demo it). If it does not run, you will get zero (No tolerance).
- Use MATLAB to do your lab assignment.
- No makeups for the Assignments are allowed.
- Please start early and ask me for help if you get stuck.

Assignment 2: Introduction to Image Processing and Computer Vision in MATLAB

Overview

Digital images are the foundation of modern Computer Vision systems. In this assignment, you will learn how images are represented as numerical arrays and how fundamental image processing techniques are used to extract meaningful information from visual data.
Using MATLAB and the Image Processing Toolbox, you will develop a complete image processing and introductory computer vision pipeline. Starting from image acquisition using a webcam, you will progressively analyze, enhance, transform, and segment images while exploring concepts that are widely used in object detection, recognition, tracking, and scene understanding.
By completing this assignment, you will be able to:

- Capture and save an image using a webcam. ***Please use your webcam to capture an image of an object of your choice (it can be your face or just pick any object with interesting texture, corners, and edges).*** Make sure the MATLAB Support Package for USB Webcams is installed (Home tab → Add-Ons → Get Hardware Support Packages).
- Read, display, and analyze digital images in MATLAB.
- Manipulate images as multidimensional arrays of numbers.
- Extract and analyze image patches and pixel neighborhoods.
- Explore and visualize RGB color channels.
- Convert color images into grayscale representations.
- Apply image smoothing filters, including averaging and Gaussian filtering.
- Explain why the coefficients of a smoothing filter should generally sum to **1**.
- Detect image edges using derivative filters and modern edge detectors.
- Explain why the coefficients of a derivative (edge) filter should sum to **0**.
- Compute and interpret image gradients, including gradient magnitude and orientation.
- Analyze image intensity distributions using histograms.
- Improve image contrast using Histogram Equalization.
- Segment an object from its background using image masking and cleanup operations.
- Compute basic object properties such as area, centroid, and bounding box.
- Gain practical experience with concepts used in modern Computer Vision systems.

Assignment Structure
This assignment consists of the following sections:

1. Image Acquisition
2. Opening and Displaying an Image
3. Interesting Point Selection and Image Patch Extraction
4. RGB Channel Analysis
5. Grayscale Image Conversion
6. Image Filtering and Gaussian Smoothing
7. Edge Detection Using Derivative Filters
8. Gradient Magnitude and Orientation Analysis
9. Image Histograms and Contrast Enhancement
10. Foreground Segmentation, Object Analysis, and Geometric Transformations

Deliverables

Submit a compressed ZIP file containing the following items:

- A single MATLAB script file named: Assignment2.m. **A template file is already shared with you.**
  The file should be divided into sections using MATLAB cell delimiters %% and add you information at the very beginning as a comment.

  %% Question 1

  …
  %% Question 2

  …

- The original image captured using your webcam: input.png
- Any additional images used in the assignment (e.g., background images).
- All output images generated by your MATLAB program.
- A PDF export of your completed MATLAB script.

Some Useful MATLAB Functions

The following MATLAB functions may be useful throughout this assignment.

Function	Description
`clear`	Delete all variables from memory
`close all`	Close all open figure windows
`clc`	Clear the Command Window
`commandhistory`	Open MATLAB command history
`webcam`	Create a webcam object
`snapshot`	Capture an image using a webcam
`imread`	Read an image file
`imwrite`	Save an image file
`imshow`	Display an image
`image`	Display image data as an image
`size`	Determine dimensions of an array
`class`	Determine the data type of a variable
`double`	Convert image data to double precision
`zeros`	Create an array of zeros
`ones`	Create an array of ones
`ginput`	Select points manually from a figure
`plot`	Plot data on a figure
`imhist`	Compute and display an image histogram
`rgb2gray`	Convert an RGB image to grayscale
`histeq`	Perform histogram equalization
`imfilter`	Apply an image filter
`fspecial`	Create predefined filters
`imgaussfilt`	Apply Gaussian smoothing
`edge`	Perform edge detection
`imgradientxy`	Compute gradient components
`imgradient`	Compute gradient magnitude and direction
`imfill`	Fill holes in binary objects
`bwareaopen`	Remove small objects from binary images
`regionprops`	Measure object properties

MATLAB Programming Notes
If you would like additional information about any MATLAB function, use:
```
help functionName
```
to view command-line documentation, or:
```
doc functionName
```
to open the full MATLAB Help Browser.

Semicolons in MATLAB
MATLAB executes one statement per line. A statement may optionally end with a semicolon (;).
```
A = zeros(100,100);
```
The semicolon suppresses output in the Command Window.
If the semicolon is omitted:
```
A = zeros(100,100)
```
MATLAB displays the result in the Command Window.
In general, it is recommended to terminate statements with semicolons to avoid unnecessary output and improve program readability.

Continuing Statements Across Multiple Lines
Long MATLAB statements may be continued across multiple lines using 3 consecutive dots:
```
...
```
For example,
```
result = function1(parameter1,...
parameter2,...
parameter3);
```

Commenting Code
Single-line comments begin with:
```
%
```
For example,
```
% Read image
I = imread('input.png');
```
Section headers should be written as:
```
%% Question 1
```
to organize the assignment into executable cells.

Academic Integrity
This is an individual assignment. All submitted work must be your own. You may discuss concepts with classmates, but sharing code or copying solutions is strictly prohibited.
Start early, test your code carefully, and ensure that every section executes correctly before submission. Untested code is often buggy, and code that does not run successfully may receive little or no credit.

Good luck, and enjoy exploring the foundations of Image Processing and Computer Vision using MATLAB!

1. Capturing and recording an image using your webcam.

(The numbers of the sections correspond to the code cells in the required assignment template)

Using MATLAB's webcam support package, capture and record an image of an object of your choice (or your face if permitted). Save the image in PNG format.
As you work through this assignment, test your commands in the MATLAB Command Window and then transfer the working solution into your Assignment2.m file. Do run `clear; close all` to clear all your variables and close all your figure windows, then re-run everything from your .m file to make sure it is complete!
Save the captured image as: **input.png** using the `imwrite()` command.

You can use the following commands:

```
cam = webcam;
preview(cam);
pause;                % move/position your object, then hit Enter in the Command Window
img = snapshot(cam);
closePreview(cam);
clear cam
imshow(img);
imwrite(img, 'input.png');
```

2. Opening and displaying an image.

Objective

In this section, you will load the image captured in Question 1, display it using MATLAB, and examine some of its basic properties.
Digital images are stored as multidimensional arrays. Before performing any image processing operation, it is important to understand the image dimensions, data type, and memory representation.

Tasks

2.1 Load the Image

Load the image you saved in Question 1 (input.png) using MATLAB. Use `imread()` command.

```
% Read image
I = imread('input.png');
```

2.2 Display the Image

Display the image in a figure window and add an appropriate title. Try to use both `imshow()` and `image()` and observe the difference between them?

2.3 Inspect the Image Properties

Determine and display:

1. Image width
2. Image height
3. Number of color channels
4. MATLAB data type (uint8, double, etc.)

Use the following commands:

```
% Determine image size
[rows, cols, channels] = size(I);

% Display information
fprintf('Image Height = %d pixels\n', rows);
fprintf('Image Width = %d pixels\n', cols);
fprintf('Color Channels = %d\n', channels);
fprintf('Data Type = %s\n', class(I));
```

2.4 Display Image Information

Print the four image properties above to the MATLAB Command Window.

3. Investigating an Interesting Point and Extracting an Image Patch

Objective

In computer vision, many algorithms operate on local neighborhoods rather than entire images. Examples include edge detection, feature detection, object recognition, and image matching.
In this question, you will select a visually interesting point in your image and examine the pixel values surrounding that location.

Tasks

3.1 Select an Interesting Point

Display your image and use MATLAB's ginput() function to select a point that contains significant visual information.

Good choices include:

- Corners
- Sharp edges
- High-contrast regions
- Object boundaries

Avoid selecting a completely smooth region with little variation. Use the following MATLAB Commands:

```
I = imread('input.png');
figure;
imshow(I);
title('Select an Interesting Point');
% Select point
[j,i] = ginput(1);
```

3.2 Record the Coordinates

Store the selected point coordinates in variables *i* and *j* and display them in the Command Window. Also, display the RGB values of the selected pixel in the Command Window. Use the following Matlab command.

```
% Round coordinates to nearest pixel
i = round(i);
```

```
j = round(j);
selectedPixel = squeeze(I(i,j,:));
fprintf('Selected Point = (%d,%d)\n',i,j);
fprintf('\nRGB Values:\n');
fprintf('Red = %d\n', selectedPixel(1));
fprintf('Green = %d\n', selectedPixel(2));
fprintf('Blue = %d\n', selectedPixel(3));
```

3.3 Extract a 10 × 10 Color Patch

Using the selected coordinates, extract a small image patch approximately: 10 × 10 × 3 centered on the chosen pixel. Display the extracted image patch. Use the following commands:

```
% Extract 10x10 patch
Ismall = I(i-5:i+4 , j-5:j+4 , :);
% Display patch
figure;
imshow(Ismall);
title('10 x 10 Image Patch');
% Display patch values
disp(Ismall);
```

3.4 Examine Pixel Values

Print the RGB values of the extracted patch in the MATLAB Command Window.
Include the output as a comment in your submitted MATLAB file.
Example:

```
% Ismall(:,:,1) =
% 221 222 215 ...
%
% Ismall(:,:,2) =
% 220 221 218 ...
%
% Ismall(:,:,3) =
% 217 216 220 ...
```

Aside: Understanding MATLAB Image Indexing

Remember that for an RGB image: I(row,column,color)
where:
I(i,j,1) % Red channel
I(i,j,2) % Green channel
I(i,j,3) % Blue channel

4. RGB Channel Analysis

Objective

A color image is typically represented using the **RGB color model**, where each pixel contains three intensity values:

- Red (R)
- Green (G)
- Blue (B)

In this exercise, you will separate the three color channels, visualize them individually, and analyze their contribution to the overall image appearance.

Tasks

4.1 Extract the Red Channel

Create an image that only contains the red information from the original image.
Display the resulting image. Use the following commands.

```
% RGB Channel Analysis
I = imread('input.png');
% Convert to double for easier processing
I = double(I)/255;
% Create Red Image
Ired = zeros(size(I));
Ired(:,:,1) = I(:,:,1);
```

4.2 Extract the Green Channel

Create an image that only contains the green information from the original image.
Display the resulting image. Use the following commands:

```
% Create Green Image
Igreen = zeros(size(I));
Igreen(:,:,2) = I(:,:,2);
```

4.3 Extract the Blue Channel

Create an image that only contains the blue information from the original image.
Display the resulting image.

4.4 Display Individual Channels as Grayscale Images

Display:
I(:,:,1)
I(:,:,2)
I(:,:,3)
using imshow().
Observe how the brightness varies across the image for each color channel. You can use the following commands:

```
% Display results
figure;
subplot(2,2,1);
imshow(I);
title('Original Image');
subplot(2,2,2);
imshow(Ired);
title('Red Channel');
subplot(2,2,3);
imshow(Igreen);
title('Green Channel');
subplot(2,2,4);
imshow(Iblue);
title('Blue Channel');
```

5. Creating a Grayscale Image

Objective

Many computer vision algorithms operate on grayscale images rather than color images because they are simpler to process and require less computational resources.

In this section, you will:

- Manually convert an RGB image into a grayscale image.
- Use MATLAB's built-in grayscale conversion function.
- Compare the results obtained from both approaches.
- Analyze the advantages and limitations of each method.

Background

An RGB image contains three color channels:

R

G

B

A simple grayscale image can be obtained by averaging the three channels (Must cast the image first as double) and then cast the final image as uint8:

$$Gray(i,j) = \frac{R(i,j) + G(i,j) + B(i,j)}{3}$$

This approach treats all colors equally.

However, the human eye does not perceive all colors with the same sensitivity. Therefore, MATLAB's built-in function uses a weighted conversion based on human visual perception.

Tasks

5.1 Create a Grayscale Image Manually

Using the RGB image from Question 4, compute a grayscale image by averaging the three color channels.

Do not use any loops.

Display the result. You can use the following commands.

```
I = imread('input.png');
% Convert to double
Id = double(I);
% Manual grayscale conversion
Igray_manual = (Id(:,:,1) + Id(:,:,2) + Id(:,:,3))/3;
% Normalize for display
Igray_manual = uint8(Igray_manual);
```

5.2 Create a Grayscale Image Using MATLAB

Use MATLAB's built-in function:

rgb2gray()

to generate another grayscale version of the image.

Display the result.

5.3 Compare the Two Results

Place both grayscale images side-by-side and compare them visually. You can use the following commands.

Do they appear identical?
Explain any observed differences.

```
figure;
subplot(1,3,1);
imshow(I);
title('Original RGB Image');
subplot(1,3,2);
imshow(Igray_manual);
title('Manual Grayscale');
subplot(1,3,3);
imshow(Igray_builtin);
title('rgb2gray()');
```

6. Image Filtering and Gaussian Smoothing

Objective
Images captured from cameras often contain noise and unwanted variations. Before performing edge detection, segmentation, or feature extraction, it is common to smooth the image using low-pass filters.
In this section, you will:

- Apply an averaging (mean) filter.
- Apply a Gaussian filter.
- Compare the results.
- Analyze the effect of smoothing on image details and noise.

Background
Image filtering is performed by moving a small matrix called a **kernel** (or filter mask) across the image.
A smoothing filter replaces each pixel with a weighted average of its neighboring pixels.
A simple averaging filter is:

$$\frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

A Gaussian filter gives more weight to pixels near the center: one example can be

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

Tasks
6.1 Create a 3×3 Averaging Filter
Create a filter that averages the values of neighboring pixels.
Apply the filter (using *imfilter* function) to the grayscale image obtained in Question 5.
Display the filtered image. You can use the following commands:

```
Igray = rgb2gray(imread('input.png'));
% Averaging filter: You can either use one of the following
avgFilter = 1/9 * ones(3,3)
```

```
%avgFilter = fspecial('average',[3 3]);

Iavg = imfilter(Igray,avgFilter);
figure;
imshow(Iavg);
title('Average Filter Result');
```

6.2 Create a Gaussian Filter

Apply a Gaussian filter to the same grayscale image.
Display the resulting image. Use the following commands:

```
% Gaussian smoothing
Igauss = imgaussfilt(Igray,2);
figure;
imshow(Igauss);
title('Gaussian Filter Result');
```

6.3 Compare Results

Display:

- Original grayscale image
- Averaged image
- Gaussian-smoothed image

using a single figure window (utilize subplot function). You can use the following commands:

```
figure;
subplot(1,3,1);
imshow(Igray);
title('Original');
subplot(1,3,2);
imshow(Iavg);
title('Average Filter');
subplot(1,3,3);
imshow(Igauss);
title('Gaussian Filter');
```

6.4 Observe the Effect of Smoothing

Zoom into an area containing edges and fine details.
Compare in your own words how each filter affects:

- Noise
- Sharp edges
- Image details

Which smoothing filter preserves edges better and why?
Include your answers as comments within the MATLAB solution script.

6.5 Why should the coefficients of a smoothing filter generally sum to 1?

- What would happen if the filter coefficients summed to 0.5 instead of 1.
- What would happen if the filter coefficients summed to 2 instead of 1.

Include your answers as comments within the MATLAB solution script.

7. Edge Detection Using Derivative Filters

Objective

Edges represent locations in an image where the intensity changes rapidly. They often correspond to:

- Object boundaries
- Corners
- Surface discontinuities
- Texture changes

In computer vision, edge information is widely used for:

- Image segmentation
- Object detection
- Feature extraction
- Recognition and tracking

In this section, you will:

- Compute image derivatives.
- Detect edges using derivative filters.
- Compare manual derivatives with Sobel and Canny edge detectors.
- Explain why edge filters must have a sum equal to zero.

Background

Consider the grayscale image as an intensity function:

$$I(i,j)$$

where:

- i= row index
- j= column index

The derivative measures how rapidly the intensity changes.

A simple approximation of the first derivative (rate of change) is:

$$I(i+1) - I(i-1)$$

which leads to the derivative filter:

[-1 0 1]

If the pixel intensities do not change, the derivative output will be close to zero.

If there is a sharp change in intensity, the derivative output will be large.

Aside: Filtering can be used to find edges in an image as specified above. We will use a simple derivative filter to accomplish this task on the Gray scale image. A derivate filter is a discrete approximation of the partial derivatives of an image. Think of the grayscale image IGray as a function mapping row/column indices i and j to inensity values, e.g. *IGray(i,j)* as a continuous function. Then the partial derivatives of this function would also be images e.g., $\partial I/\partial i\ (i,j) = I_i(i,j)$ gives the partial derivative in the vertical direction at each point in the image, and $\partial I/\partial j\ (i,j) = I_j$ gives the partial derivative in the horizontal direction.

While I might have been a continuous function somewhere in the camera that took the picture, all we have are discrete values. So instead of computing the "true" derivative, we will compute an approximation. The classic approximation of a derivative is $I_i \approx (I(i+di)-I(i))/di$. Instead of this we will use *I(i+1) - I(i-1)*. (This result will have half the magnitude of the true derivative, but that is not generally a concern for us. We are most interested in the relative values of the derivative from one spot in the image to another.)

Tasks

7.1 Compute the Horizontal Derivative

Define and apply the derivative filter:
Dx = [-1 0 1];
to the smoothed grayscale image obtained in Question 6.
Display the resulting image. Use the following commands:

```
% Use Gaussian-smoothed image from Question 6
I = Igauss;
% Horizontal derivative
Dx = [-1 0 1];
dIdj = imfilter(double(I),Dx,'replicate');
```

7.2 Compute the Vertical Derivative

Define:
Dy = Dx'; or Dx = [-1; 0; 1];
and apply it to the image.
Display the result. Use the following commands:

```
% Vertical derivative
Dy = Dx';
dIdi = imfilter(double(I),Dy,'replicate');
```

7.3 Visualize Both Derivatives

Display:

- Original grayscale image
- Horizontal derivative image
- Vertical derivative image

in a single figure window. Use subplot function as shown below.

```
figure;
subplot(1,3,1);
imshow(I,[]);
title('Input Image');
subplot(1,3,2);
imshow(dIdj,[]);
title('Horizontal Derivative');
subplot(1,3,3);
imshow(dIdi,[]);
title('Vertical Derivative');
```

7.4 Compare with Sobel Edge Detection

Apply MATLAB's Sobel detector. You can use the following command:

```
BWsobel = edge(I,'sobel');
```

Display the results you obtained:

```
figure;
imshow(BWsobel);
title('Sobel Edge Detection');
```

7.5 Compare with Prewitt Edge Detection

Apply MATLAB's Prewitt detector and display the results. Use the following commands:

```
BWprewitt = edge(I,'prewitt');
figure;
imshow(BWprewitt);
title('Prewitt Edge Detection');
```

7.6 Compare with Canny Edge Detection

Canny is one of the most commonly used edge detectors. Apply MATLAB's Canny detector and display the results. Use the following commands:

```
BWcanny = edge(I,'canny');
figure;
imshow(BWcanny);
title('Canny Edge Detection');
```

7.7 Compare All Methods

Display all edge images together. Use subplot.

```
figure;
subplot(2,2,1);
imshow(abs(dIdj),[]);
title('Derivative X');
subplot(2,2,2);
imshow(abs(dIdi),[]);
title('Derivative Y');
subplot(2,2,3);
imshow(BWsobel);
title('Sobel');
subplot(2,2,4);
imshow(BWcanny);
title('Canny');
```

7.8 Explain why the coefficients of a derivative (edge) filter should sum to 0?

Hint: Consider what should happen when the image intensity is constant.

Include your answers as comments within the MATLAB solution script.

8. Gradient Magnitude and Orientation Analysis

Objective

In Question 7, you computed the image derivatives:

dIdi

dIdj

which represent the rate of intensity change along the image rows and columns, respectively.

In this section, you will combine these derivatives to compute:

- Gradient Magnitude (Edge Strength)
- Gradient Orientation (Edge Direction)

These quantities are used extensively in many computer vision algorithms, including:

- Harris Corner Detection
- SIFT Features
- HOG Features
- Object Recognition
- Image Matching

Background

Given:

$$I_i = \frac{\partial I}{\partial i}$$

and

$$I_j = \frac{\partial I}{\partial j}$$

the gradient magnitude is defined as:

$$M(i,j) = \sqrt{I_i(i,j)^2 + I_j(i,j)^2}$$

The gradient orientation is defined as:

$$\theta(i,j) = atan2\left(I_j,(i,j)I_i(i,j)\right)$$

where:

- Large magnitude values correspond to strong edges.
- Orientation indicates the direction of the maximum intensity change.

Tasks

8.1 Compute Gradient Magnitude

Using the derivative images obtained in Question 7, compute the gradient magnitude image. You can use the following commands:

```
% Convert derivatives to double
Ii = double(dIdi);
Ij = double(dIdj);
% Magnitude
M = sqrt(Ii.^2 + Ij.^2);
```

8.2 Compute Gradient Orientation

Compute the gradient orientation image using: atan2()

You can use the following command:

```
% Orientation
Theta = atan2(Ij,Ii);
```

8.3 Display the Results

Display:

1. Original grayscale image
2. Gradient magnitude image
3. Gradient orientation image

using subplots.

Use the following commands:

```
figure;
subplot(1,3,1);
imshow(Igray);
title('Grayscale Image');
```

```
subplot(1,3,2);
imshow(M,[]);
title('Gradient Magnitude');
subplot(1,3,3);
imshow(Theta,[]);
title('Gradient Orientation');
colormap jet;
colorbar;
```

8.4 MATLAB provides built-in gradient functions.

Compare your results with:

```
[Gx,Gy] = imgradientxy(Igray);
[MagBuiltIn,ThetaBuiltIn] = imgradient(Gx,Gy);
```

Display the results and compare them with your manually computed magnitude and orientation images above.

8.5 Many modern feature descriptors analyze edge orientation distributions.

Create a histogram of all orientation values. Use the following commands:

```
figure;
histogram(rad2deg(Theta(:)),36);
xlabel('Orientation (Degrees)');
ylabel('Frequency');
title('Gradient Orientation Histogram');
```

8.6 Analyze a Selected Pixel

Use the same point selected in Question 3.

Report:

- dIdi
- dIdj
- Magnitude
- Orientation

at that location.

You can use the following commands (first remember to read the values of i and j from Question 3)

```
fprintf('dIdi = %.2f\n', Ii(i,j));
fprintf('dIdj = %.2f\n', Ij(i,j));
fprintf('Magnitude = %.2f\n', M(i,j));
fprintf('Orientation = %.2f radians\n', Theta(i,j));
fprintf('Orientation = %.2f degrees\n', rad2deg(Theta(i,j)));
```

9. Image Histograms and Contrast Enhancement

Objective

Histograms provide a compact summary of the intensity distribution within an image.

In this section, you will:

- Compute the histogram of a grayscale image.
- Interpret the histogram.
- Apply Histogram Equalization.

- Compare the original and enhanced histograms.
- Analyze how contrast enhancement affects image quality.

Background

A grayscale image contains intensity values ranging from:
0 → Black
255 → White
A histogram shows how many pixels exist at each intensity level.
Histograms are useful for:

- Image enhancement
- Thresholding
- Segmentation
- Exposure analysis
- Feature extraction

Tasks

9.1 Display the Histogram of the Original Grayscale Image

Using the grayscale image from Question 5, display its intensity histogram. Use the following commands:

```
% Image Histograms and Contrast Enhancement
Igray = rgb2gray(imread('input.png'));
% Original histogram
figure;
imhist(Igray);
title('Histogram of Original Grayscale Image');
```

9.2 Interpret the Histogram

Based on the histogram, determine whether your image appears:

- Dark
- Bright
- High-contrast
- Low-contrast

Briefly explain your reasoning.
Include your answers as comments within the MATLAB solution script.

9.3 Apply Histogram Equalization

Use MATLAB's histogram equalization function:
histeq()
to improve image contrast.
Display the equalized image. Use the following commands:

```
% Equalized image
Ieq = histeq(Igray);
figure;
imshow(Ieq);
title('Histogram Equalized Image');
```

9.4 Display the Equalized Histogram

Plot the histogram of the equalized image.

Compare it with the original histogram. You can use the following commands:

```
%Equalized Histogram
figure;
imhist(Ieq);
title('Histogram After Equalization');
```

9.5 Compare Original and Enhanced Images

Display:

- Original grayscale image
- Equalized image

side-by-side.

Discuss the visual differences. You can compute mean intensity and standard deviation of these images.

Include your answers as comments within the MATLAB solution script.

You can use the following commands:

```
figure;
subplot(2,2,1);
imshow(Igray);
title('Original Image');
subplot(2,2,2);
imhist(Igray);
title('Original Histogram');
subplot(2,2,3);
imshow(Ieq);
title('Equalized Image');
subplot(2,2,4);
imhist(Ieq);
title('Equalized Histogram');
fprintf('Original Mean Intensity = %.2f\n', mean(Igray(:)));
fprintf('Equalized Mean Intensity = %.2f\n', mean(Ieq(:)));
fprintf('Original Std Deviation = %.2f\n', std(double(Igray(:))));
fprintf('Equalized Std Deviation = %.2f\n', std(double(Ieq(:))));
```

10. Foreground Segmentation, Object Analysis, and Geometric Transformations

Objective

One of the fundamental goals of computer vision is to separate objects of interest from their background. This process is known as **image segmentation**.

In this section, you will:

- Extract an object from the background.
- Create a binary mask.
- Clean the mask using morphological operations.
- Display the segmented object.
- Compute basic object properties.

This exercise combines many concepts learned earlier in the assignment, including grayscale conversion, filtering, edge information, and image analysis.

Background

A **binary mask** is an image containing only two values:

0 = Background

1 = Foreground Object

Using a mask allows us to isolate an object from the rest of the image.

To improve the quality of the mask, MATLAB provides morphological operations such as:

`imfill()`

`bwareaopen()`

which remove noise and fill holes inside objects.

Tasks

10.1 Capture a Background Image

Capture **a second image** that contains **only the background without the object**.

Save the image as:

background.png

10.2 Compute Background Subtraction

Compute the difference between:

- Original image
- Background image

to identify the object.

Use the following commands

```
% Foreground Segmentation
Iobj = imread('input.png');
Ibg = imread('background.png');
 D = abs(double(Iobj) - double(Ibg));
figure;
imshow(uint8(D));
title('Difference Image');
```

10.3 Create a Binary Mask

Convert the difference image to grayscale and generate a binary mask. Use the code below.

```
Dgray = rgb2gray(uint8(D));
mask = Dgray > 25;
figure;
imshow(mask);
title('Initial Binary Mask');
```

You may experimentally adjust the threshold value (we used 25 above).

10.4 Improve the Mask

Remove noise and fill holes. You can use the code below.

```
mask = imfill(mask,'holes');
mask = bwareaopen(mask,100);
figure;
imshow(mask);
title('Cleaned Binary Mask');
```

10.5 Extract the Object

Use the mask to keep only the object pixels. Use the code below.

```
segmented = Iobj;
segmented(repmat(~mask,[1 1 3])) = 0;
figure;
imshow(segmented);
title('Segmented Object');
```

10.6 Compute Object Properties

Use MATLAB's region analysis tools. Use the code below

```
stats = regionprops(mask,...
'Area',...
'Centroid',...
'BoundingBox');
disp(stats);
```

10.7 Display Bounding Box

Draw the object bounding box. Use the code below

```
figure;
imshow(Iobj);
hold on;
rectangle('Position',stats(1).BoundingBox, 'EdgeColor','r',
'LineWidth',2);
plot(stats(1).Centroid(1), stats(1).Centroid(2),'b*');
title('Object Bounding Box and Centroid');
```

10.8 Geometric Transformations of the Segmented Object

Geometric transformations are fundamental operations in computer vision and image analysis. They are commonly used in image registration, object recognition, data augmentation for machine learning, and image alignment.

Using the segmented object image *segmented* obtained in Question 10.5, perform the following geometric transformations and display each result.

Using your segmented object image: *segmented*

perform the following operations:

Rotate the segmented object image by 60 degrees clockwise. Please use both of the following methods:

Method 1: Using imrotate()

```
Jrot = imrotate(segmented,-60);
figure;
imshow(Jrot);
title('60° Clockwise Rotation');
```

Note: MATLAB uses positive angles for counterclockwise rotations and negative angles for clockwise rotations.

Method 2: Using a General Affine Transformation
This approach is more general and can later be extended to scaling, shearing, and translations.

```
theta = deg2rad(-60);
R = [ cos(theta) sin(theta) 0;
-sin(theta) cos(theta) 0;
0 0 1 ];
tform = affinetform2d(R);
Jrot2 = imwarp(segmented,tform);
figure;
imshow(Jrot2);
title('60° Clockwise Rotation Using Affine Transform');
```